



The Art of Guard

Part 2

Let's explore SELinux modes, and learn a few commands that will help in understanding SELinux policies better.

In Part I of this series, we discussed the basics of host security through traditional security mechanisms (DAC) and newer efficient mechanisms (MAC). We also understood that the right combination of both the Access Control Methods eventually leads to an effective security policy that can be applied to any host.

We learnt that applying security to an operating system as a whole was far more practical and efficient than applying security to each individual application running on the operating system, and that SELinux was one such mechanism. SELinux achieves MAC objectives by applying a 'Security Context' to subjects and objects, and by controlling access of subjects to objects based on access control rules.

In Part I, we also learnt to enable SELinux on Red Hat Enterprise Linux. The steps are briefly summarised below:

- a) Edit `/etc/sysconfig/selinux` and make sure it contains the lines:

```
SELINUX=permissive
SELINUXTYPE=targeted
```

- b) Reboot and check.

In this article, we will explore Permissive and Enforcing Modes. We will also learn a few SELinux commands that will help in understanding SELinux policies better.

Understanding SELinux modes

Irrespective of the policy or the rules implemented through SELinux Type Enforcement, there are three modes of operation for SELinux:

- a) Disabled
- b) Permissive
- c) Enforcing

Disabled mode implies that SELinux is disabled and

not implemented on the host. This has been the most common choice in installations seen by me. Hopefully, by the end of this series, we shall be able to bring about a change in that practice by encouraging more system administrators to adopt SELinux.

Permissive mode is similar to Debugging Mode. In Permissive Mode, SELinux policies and rules are applied to subjects and objects, but actions (for example, Access Control denials) are not effected. The biggest advantage of Permissive Mode is that log files and error messages are generated based on the SELinux policy implemented.

In other words, if the SELinux policy would prevent the httpd subject (Apache Web server) from accessing the object folder `/webdata` on my system, implementing SELinux in Permissive Mode would let the Apache Web server access the folder `/webdata` but log a denial in the log files.

This error logging informs the system administrator that if SELinux is activated in the Enforcing Mode, the httpd subject would be disallowed access to the `/webdata` folder on my system.

Permissive Mode is the initiating point for all those wanting to explore the world of Type Enforcement through SELinux. Without blocking access to your favourite programs such as OpenOffice.org, Evolution, etc, it provides you with enough debugging information to fine tune your policy before deploying it on your system.

Enforcing Mode, as the name signifies, is SELinux in action. All production systems, when hardened, should enable SELinux in Enforcing Mode. SELinux through Access Controls does have a minor performance overhead, but compared to the advantages that it brings to the table, I am sure it will soon become the norm to implement SELinux on production servers.